

Relatório 12 – Pipeline de Dados

Edryck Freitas Nascimento

1. Introdução

O objetivo desta aula foi aprender sobre alguns conceitos importantes sobre projetar *pipeline* de dados automatizados. A tarefa consistia em assistir o minicurso “*Apache Airflow: The Hands-On Guide*” e, como prática, desenvolver uma *pipeline* de dados automatizados apresentando o conhecimento adquirido.

2. Desenvolvimento

Como as aulas do minicurso foi apresentado a construção de uma *pipeline* de dados, onde era realizada a extração de dados de câmbio de moedas, pensei em fazer algo parecido, porém, com ações da B3 (Brasil, Bolsa, Balcão; Bolsa de valores oficial do Brasil).

Inicialmente, encontrei uma biblioteca *Python*, o *yfinance*, ela é uma biblioteca de *open-source* que permite acessar dados financeiros históricos e em tempo real, disponibilizado pelo *Yahoo Finance*. Ela retorna os dados diretamente em *DataFrames*. Depois, descobri que o *Yahoo Finance* bloqueava o *yfinance* e como solução para isso utilizei o *yfinance_cache*, ele atua como um *wrapper* para o *yfinance* e evita as requisições repetidas ao servidor, consequentemente acelera a coleta de dados e reduz o risco de bloqueios por excesso de solicitações. No geral, as funções do *yfinance* e *yfinance_cache* são as mesmas.

- **Obtenção de Dados:** Inicialmente, utilizei o *yfinance_cache* para obter os dados das ações, na qual passei os *tickers* (“id” das ações) dentro de uma lista. Nesta tarefa, ele coleta os dados em um período de 1 dia, pega do dia atual, e armazena em um *DataFrame*.
- **Preparação dos Dados:** Os dados armazenados no *DataFrame* passam por duas tarefas, deveriam ser em paralelo, porém, não soube como fazer isso, uma que vai salvar em um arquivo CSV e outra que vai gerar um resumo do dia para cada ação.
- **Final da Pipeline:** Ao final da *pipeline*, é enviado um email e uma mensagem no Discord, indicando que a *pipeline* foi finalizada, no caso de ter sido finalizada com sucesso.

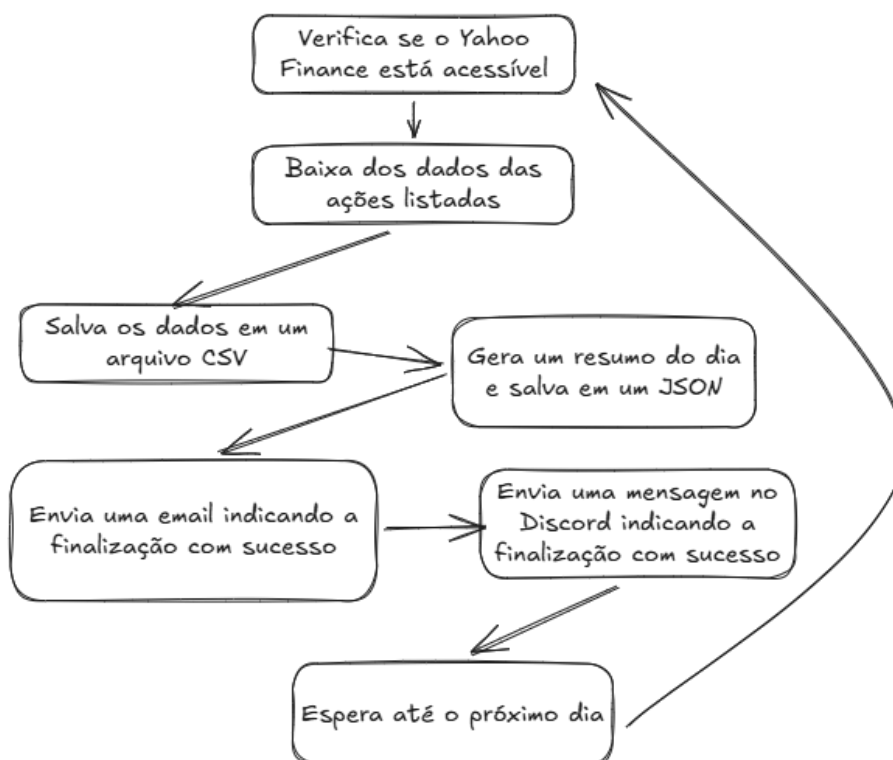


Figura 1. *Pipeline* da dados B3. (autoria própria)

2.1. Configurando

Antes de iniciar, é necessário completar partes de alguns arquivos, eu não estou enviando junto porque são algumas informações pessoais, como senhas, etc. Os principais são os arquivos: `airflow.cfg` e `docker-compose.yml`.

2.1.1. Configurando o SMTP

Antes de ir para os dois arquivos citados anteriormente, é necessário pegar uma senha gerada pelo Google para configurar a parte de mandar email da *pipeline*. Para isso entre neste *link*:

<https://security.google.com/settings/security/apppasswords>, entrando nele, vai pedir a senha da sua conta do Google e você precisa ter habilitado a autenticação de dois fatores. Você precisa adicionar o nome do aplicativo, no meu caso adicionei o nome da minha *pipeline*, vai gerar uma senha (neste formato: xxxx xxxx xxxx xxxx), antes de fechar salve essa senha, ele não abre de volta.

Google Conta

← Senhas de app

As senhas de app são menos seguras do que usar apps e serviços atualizados com padrões de segurança modernos. Antes de criar uma senha, verifique se o app precisa disso para fazer login.

[Saiba mais](#)

Suas senhas de app

B3-Pipeline-de-Dados	Criada em 26 de mar., usada pela última vez em 30 de mar.	
----------------------	---	---

Para criar uma senha específica para um app, digite o nome dele abaixo...

Criar

Figura 2. Tela de Senhas de app. (autoria própria)

2.1.2. docker-compose.yml

Agora que tem a senha dá para começar completar estes arquivos. Neste arquivo, preencha os campos:

```
version: '3.8'
D>Run All Services
services:
  D>Run Service
  airflow:
    build: ./docker/airflow
    restart: always
    container_name: airflow
    depends_on:
      - mysql
    volumes:
      - ./mnt/airflow/dags:/usr/local/airflow/dags
      - ./mnt/data:/usr/local/airflow/data
    ports:
      - "8080:8080"
    environment:
      - AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=mysql+mysqldb://airflow:airf
      - AIRFLOW__CORE__LOAD_EXAMPLES=False
      - AIRFLOW__SMTP__SMTP_HOST=smtp.gmail.com
      - AIRFLOW__SMTP__SMTP_USER=seu_email
      - AIRFLOW__SMTP__SMTP_PASSWORD=xxxxx xxxxx xxxxx xxxxx
      - AIRFLOW__SMTP__SMTP_PORT=587
      - AIRFLOW__SMTP__SMTP_MAIL_FROM=seu_email
      - AIRFLOW__SMTP__SMTP_SSL=False
      - AIRFLOW__SMTP__SMTP_STARTTLS=True
    healthcheck:
      test: [ "CMD", "nc", "-z", "airflow", "8080" ]
      timeout: 45s
      interval: 10s
      retries: 10
    dns:
```

Figura 3. Modelo de docker-compose.yml. (autoria própria)

Em *user* e *smtp_mail* adicione seu email, em *smtp_password* coloque a senha gerada em Senha de app no Google.

2.1.3. airflow.cfg

Agora no arquivo *airflow.cfg*, procure por *[smtp]* e *[webserver]*.

Em *[smtp]*:

```
[smtp]
smtp_host = smtp.gmail.com
smtp_starttls = True
smtp_ssl = False
smtp_user = seu_email
smtp_password = xxxx xxxx xxxx xxxx
smtp_port = 587
smtp_mail_from = seu_email
smtp_timeout = 30
smtp_retry_limit = 5
```

Figura 4. Modelo do arquivo *airflow.cfg*. (autoria própria)

Em *[smtp]*, altere o *user* e *email_from* para o seu email e em *password* altere para aquela senha do Senhas de app do Google.

Em *[email]*, procure por *secret_key*, gere uma senha e atribua ela para *secret_key*.

2.1.4. Subindo o Contêiner

No terminal, inicia com o comando *docker-compose up -d --build*, ele vai subir e construir o contêiner. A primeira vez é demorado.

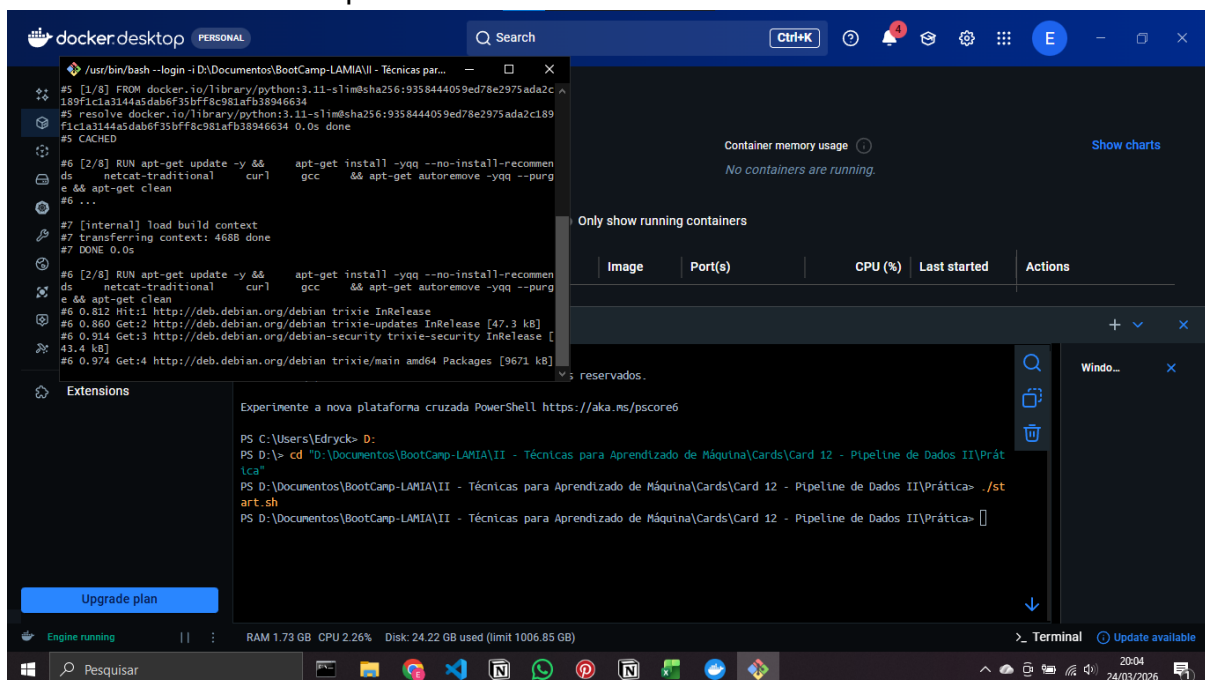


Figura 5. Subindo o contêiner. (autoria própria)

No meu caso, chamei o arquivo *start.sh*, dentro dele está o mesmo comando, só para economizar digitar mais coisas.

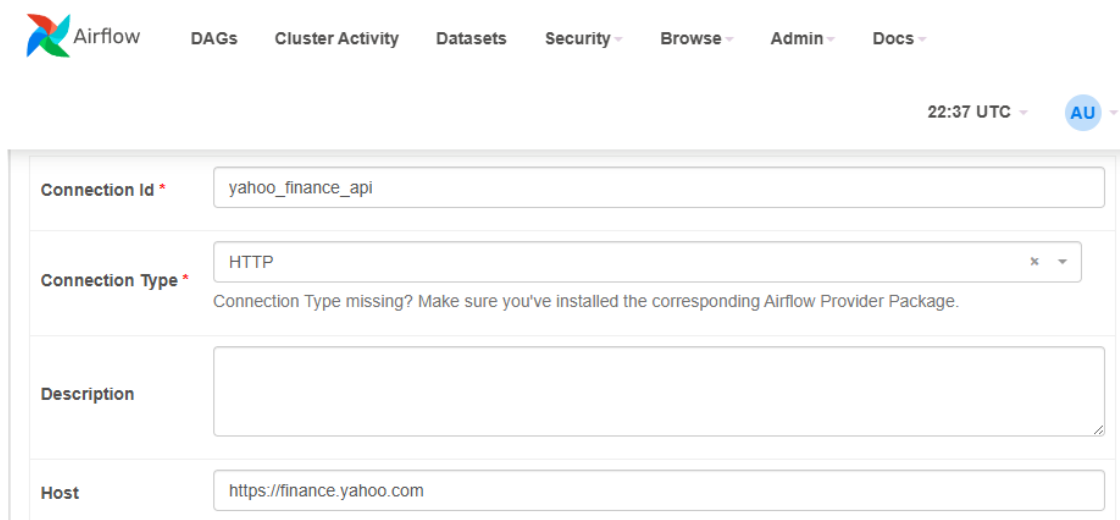
Se tudo deu certo, use o comando `docker exec -it airflow bash`, em seguida entra no *Airflow*, no navegador digite: <http://localhost:8080>. Para login o *user/password* é *admin/admin*.

2.1.5. Criando conexões e variáveis

Com o *Airflow* funcionando, será necessário criar as conexões e as variáveis necessárias, basicamente, vai precisar de 3 conexões e 2 variáveis:

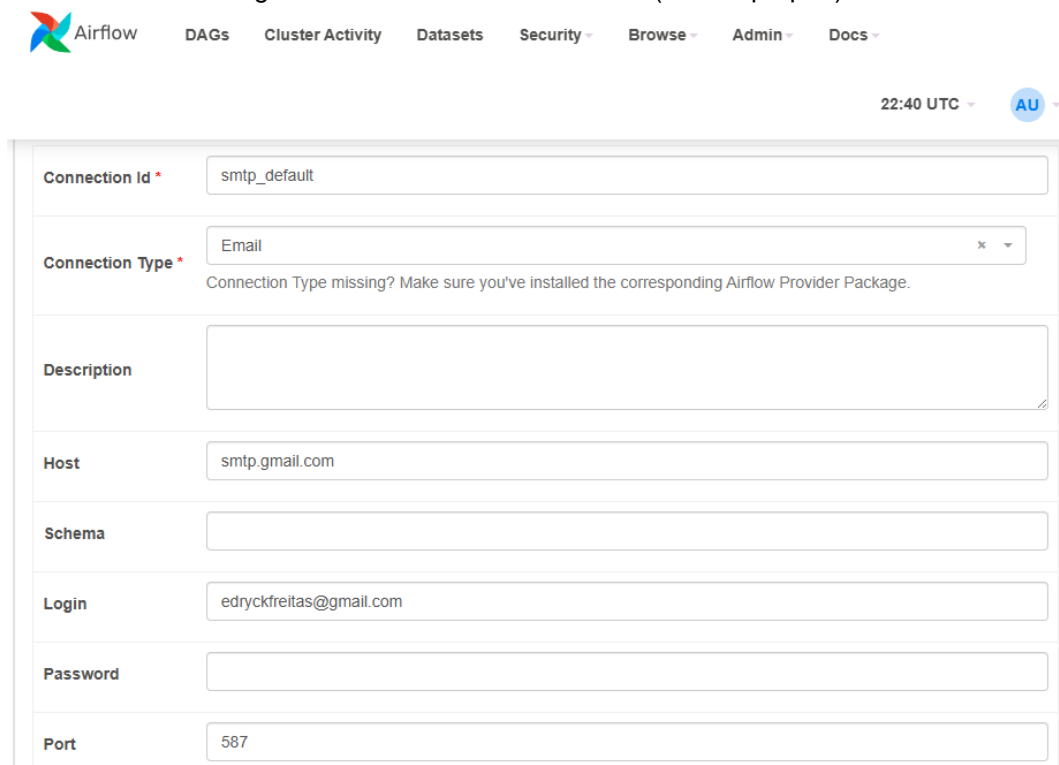
- **Conexões:** *smtp_default*, *yahoo_finance* e *discord_conn*.
- **Variáveis:** *webhook_id* e *webhook_token*.

Para as conexões, vai em Admin → Connections → Botão de criar nova conexão (botão com um mais), crie a *smtp_default* e *yahoo_finance* primeiro.



The screenshot shows the Airflow Admin interface with the 'Add Connection' form. The 'Connection Id' is 'yahoo_finance_api', the 'Connection Type' is 'HTTP', and the 'Host' is 'https://finance.yahoo.com'. The 'Description' field is empty. The top navigation bar includes 'Airflow', 'DAGs', 'Cluster Activity', 'Datasets', 'Security', 'Browse', 'Admin', and 'Docs'. The top right shows the time '22:37 UTC' and a user profile 'AU'.

Figura 6. Conexão *Yahoo Finance*. (autoria própria)



The screenshot shows the Airflow Admin interface with the 'Add Connection' form. The 'Connection Id' is 'smtp_default', the 'Connection Type' is 'Email', and the 'Host' is 'smtp.gmail.com'. The 'Description' field is empty. The 'Login' field contains 'edryckfreitas@gmail.com' and the 'Port' field contains '587'. The top navigation bar includes 'Airflow', 'DAGs', 'Cluster Activity', 'Datasets', 'Security', 'Browse', 'Admin', and 'Docs'. The top right shows the time '22:40 UTC' and a user profile 'AU'.

Figura 7. Conexão SMTP. (autoria própria)

Salva as duas acima e vamos prosseguir para a configuração do *webhook*. Para ele, se quiser criar um canal novo no servidor do *Discord*, precisa ir para as configuração do canal: Editar canal (ícone de engrenagem no canal) → Integrações → *Webhooks* → Novo *Webhook*, nele você troca o nome, se quiser e o ícone de perfil... se quiser, e confere se em Canal está no que você vai adicionar. Por último, copie a URL do *webhook* (tem literalmente um botão escrito isso).

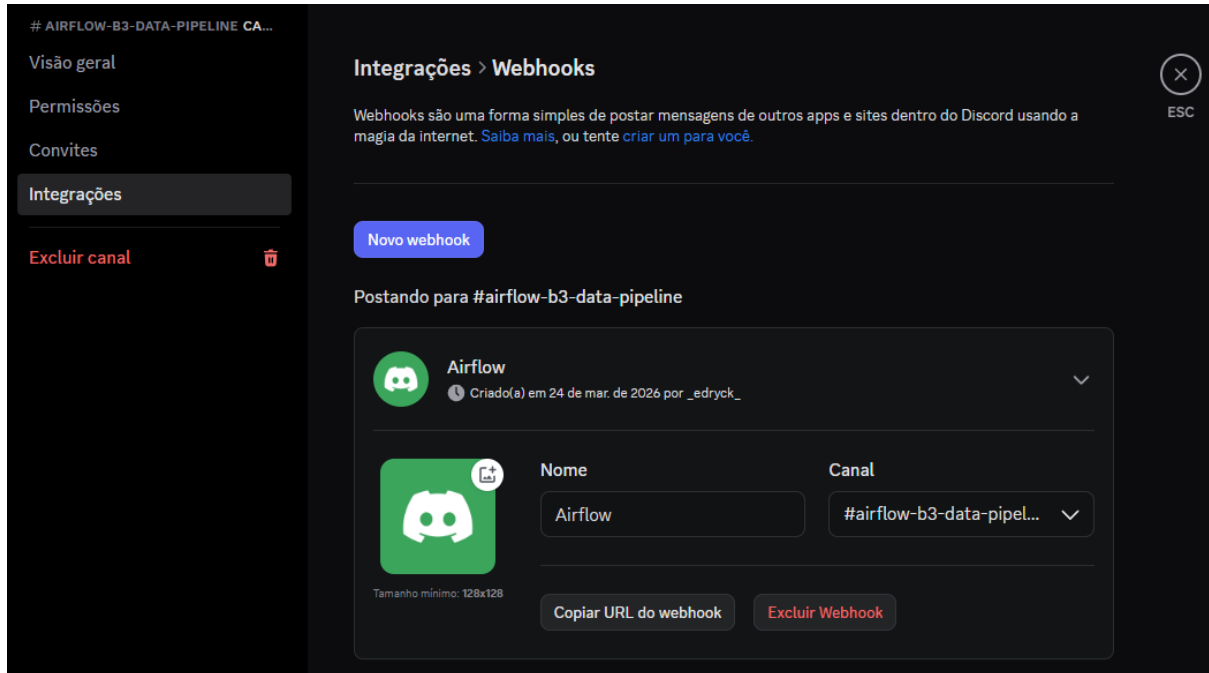


Figura 8. *Webhooks*. (autoria própria)

O formato da URL é o seguinte:

<https://discordapp.com/api/webhooks/<id>/<token>>

Para a conexão, você precisar do “https://discordapp.com/”, copie e cole em *host*, complete conforme imagem e salve.

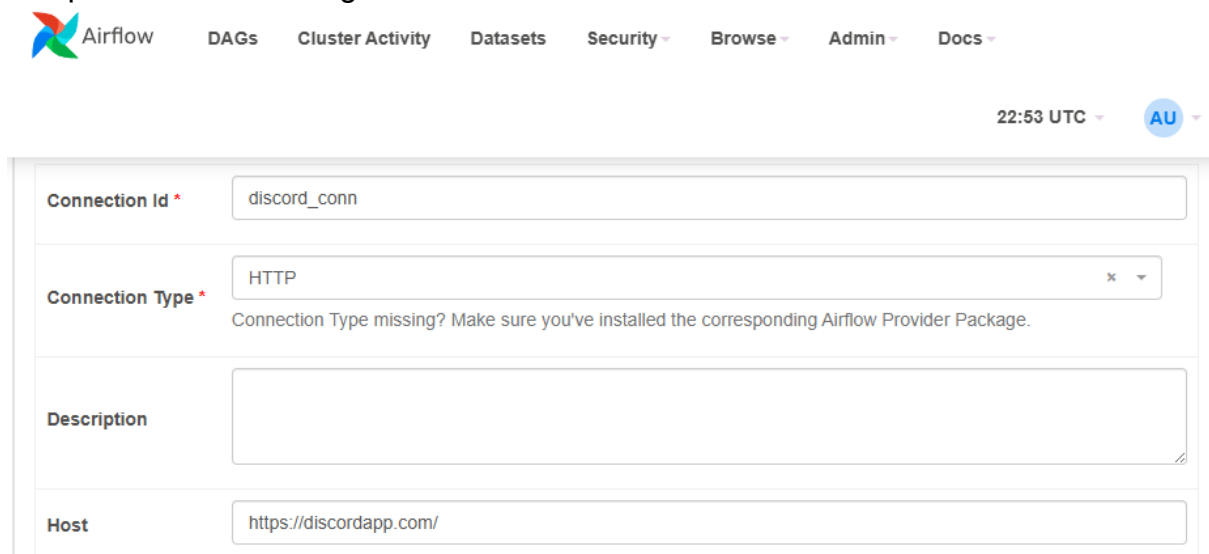
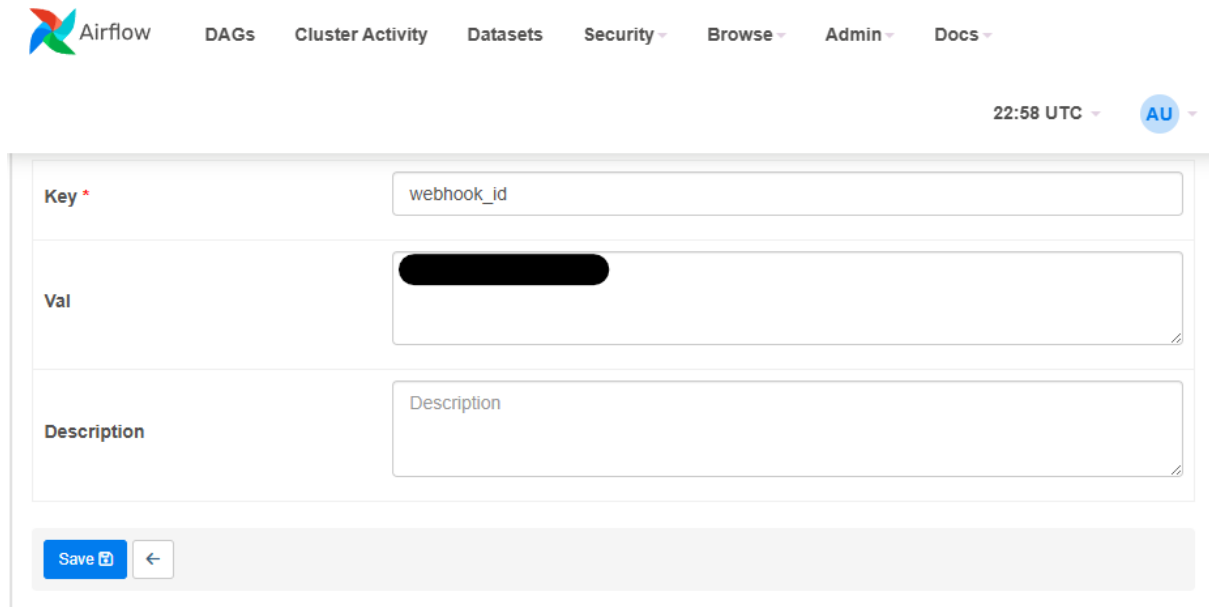


Figura 9. Criando a conexão do *Discord*. (autoria própria)

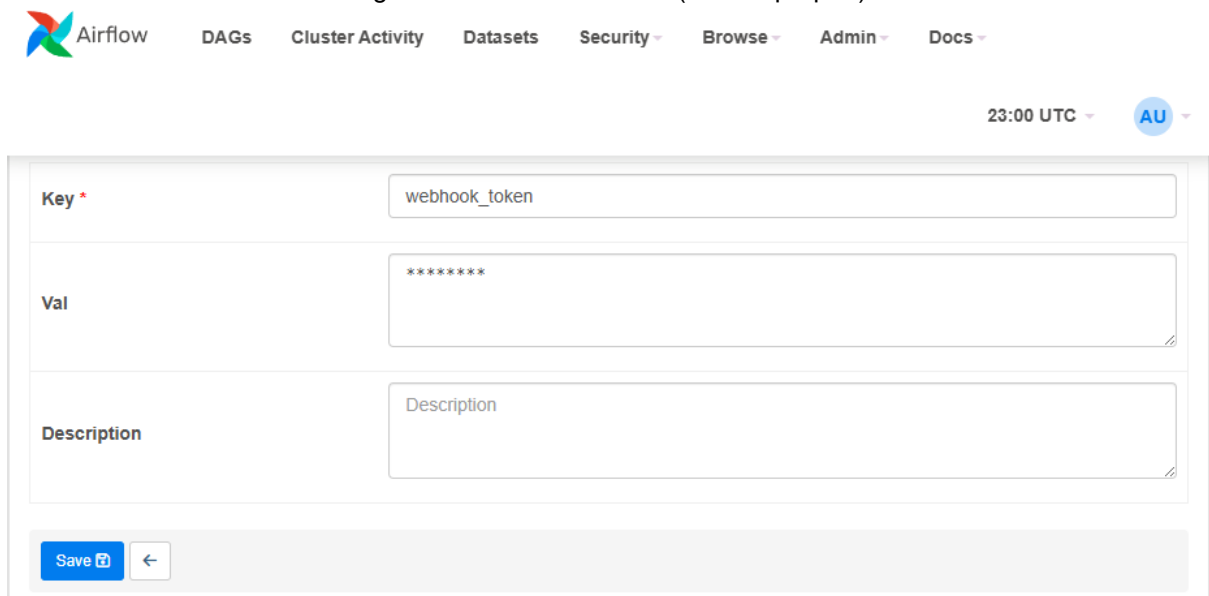
Agora as variáveis, precisa criar duas, uma é o id do *webhook* e a outra é o *token*, para isso vá em *Admin* → *Variables* → Criar nova (botão com ícone de +). Em valor

cole a parte de *<id>* e criando a outra, no valor, coloque o *<token>*, nomeie a do *id* de *webhook_id* e a do *token* de *webhook_token*.



The image shows the Airflow web interface for configuring a new webhook. The top navigation bar includes links for Airflow, DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The current time is 22:58 UTC and the user is 'AU'. The form has three fields: 'Key' with the value 'webhook_id', 'Val' which is redacted with a black box, and 'Description' which is empty. At the bottom, there are 'Save' and 'Back' buttons.

Figura 10. ID do *Webhook*. (autoria própria)



The image shows the Airflow web interface for configuring a new webhook token. The top navigation bar is the same as in Figure 10. The current time is 23:00 UTC and the user is 'AU'. The form has three fields: 'Key' with the value 'webhook_token', 'Val' with the value '*****', and 'Description' which is empty. At the bottom, there are 'Save' and 'Back' buttons.

Figura 11. *Token* do *Webhook*. (autoria própria)

Depois de Salvar os dois, vamos para a penúltima parte: os testes.

2.2. Testes

Agora que o código estava pronto, as conexões criadas e as variáveis também, era só testar as *tasks*.

Os *Ids* das *tasks* são:

- `is_yahoo_finance_available`
- `download_b3_data`

- save_to_csv
- generate_summary
- sending_email_notification
- sending_discord_message

No terminal, para testar elas, use o comando:
airflow tasks testb 3_data_pipeline << id-da-task >> 2026-01-01

Não altere a ordem.

```
Terminal
340da90c27ea prtica-airflow "/start-airflow.sh" About a minute ago Up About a minute (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp airflow
PS D:\Documentos\BootCamp-LAMIA\II - Técnicas para Aprendizagem de Máquina\Cards\Card 12 - Pipeline de Dados II\Prática> docker ps>
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
340da90c27ea prtica-airflow "/start-airflow.sh" 4 minutes ago Up 4 minutes (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp airflow
PS D:\Documentos\BootCamp-LAMIA\II - Técnicas para Aprendizagem de Máquina\Cards\Card 12 - Pipeline de Dados II\Prática> docker exec -it airflow bash
airflow@340da90c27ea:/$ airflow tasks test b3_data_pipeline is_yahoo_finance_available 2026-01-01
```

Figura 12. Testando as *tasks*. (autoria própria)

```
Terminal
it in a 'StringIO' object.
WEGE3.SA: R$50.08 ▲1.03%
ABEV3.SA: R$15.13 ▲0.87%
VALE3.SA: R$81.34 ▲0.74%
ITUB4.SA: R$42.40 ▲0.26%
PETR4.SA: R$50.17 ▲0.2%
BBAS3.SA: R$22.74 ▲0.09%
BBDC4.SA: R$18.80 ▼0.0%

Maior alta: WEGE3.SA (1.03%)
Maior queda: BBDC4.SA (0.0%)
```

Figura 13. Resultado da *task* de gerar o resumo do dia¹. (autoria própria)

```
Terminal
[message] AIRFLOW_CTX_EXECUTION_DATE='2026-01-01T00:00:00+00:00' AIRFLOW_CTX_TRY_NUMBER='1' AIRFLOW_CTX_DAG_RUN_ID='***_temporary_run_2026-03-26T19:35:00.554432+00:00_'
[2026-03-26T21:32:52.752+0000] [http.py:169] INFO - Calling HTTP method
[2026-03-26T21:32:52.764+0000] [base.py:83] INFO - Using connection ID 'discord_conn' for task execution.
[2026-03-26T21:32:52.790+0000] [base.py:83] INFO - Using connection ID 'discord_conn' for task execution.
[2026-03-26T21:32:53.559+0000] [taskinstance.py:1138] INFO - Marking task as SUCCESS dag_id=b3_data_pipeline task_id=sending_discord_message execution_date=20260101T000000 start_date= end_date=20260326T213253
airflow@50199d3bffa:/s$

RAM 7.64 GB CPU 6.81% Disk: 28.49 GB used (limit 1006.85 GB)
```

Figura 14. Resultado da última *task*, a de mandar a mensagem no *Discord*. (autoria própria)

¹ OBS.: Esse resumo aparece no terminal ao executar o teste da *task* “generate_summary”.

2.3. Iniciando o DAG

Para iniciar o DAG, abra novamente o *Airflow* (<http://localhost:8080> no navegador) e em DAGs, localize o `b3_data_pipeline` e na coluna *Actions* clique em Play (▶). Enquanto o contêiner estiver aberto, mesmo em segundo plano, a DAG vai executar diariamente.

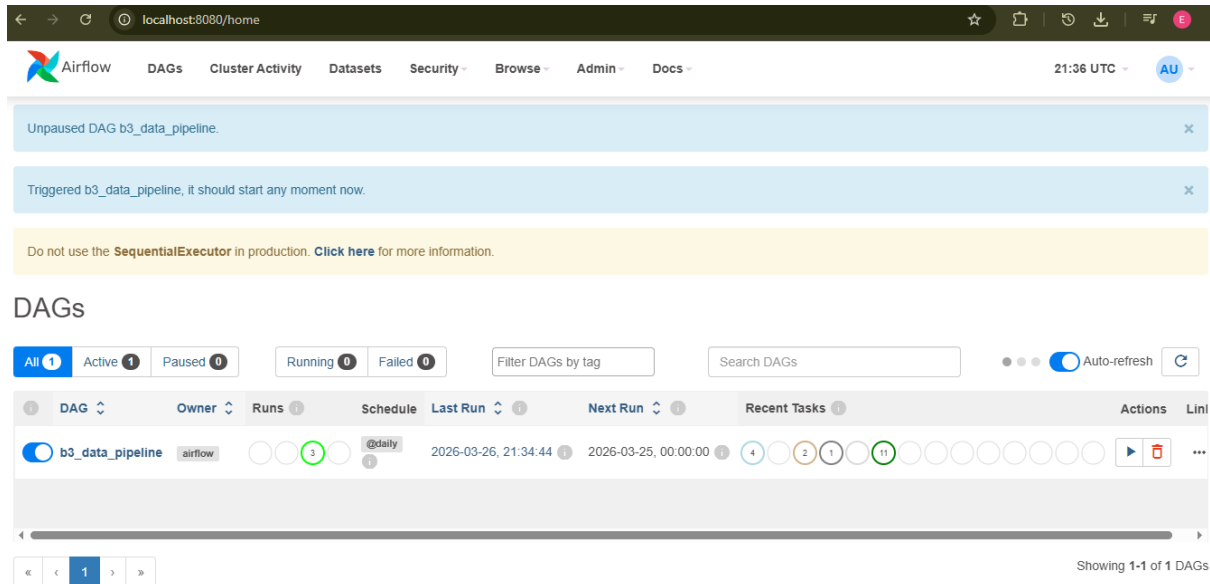


Figura 13. DAG sendo executada. (autoria própria)

3. Conclusão

A criação de uma *pipeline* de dados automatizada é algo que essencial para aumentar a eficiência da extração, preparação e armazenamento dos dados que posteriormente serão utilizados treinar um modelo. O *Docker* garante a execução da DAG em qualquer máquina. Tive dificuldades neste *Card* para executar os códigos da aula, por estar bem desatualizada, e encontrar os módulos corretos na documentação do *Airflow*. Finalmente, a *pipeline* está pronta, toda vez que ela finalizar (diariamente) será enviada uma mensagem no *Discord* e um *email*.

4. Referências

- Lamberti, M. **Apache Airflow: The Hands-On Guide**. Disponível em: <https://www.udemy.com/course/the-ultimate-hands-on-course-to-master-apache-airflow/>
- Airflow. **Documentation**. Disponível em: <https://airflow.apache.org/docs/>
- Bland, G. **yfinance Library – A Complete Guide**, 03 nov. 2020. Disponível em: <https://algotrading101.com/learn/yfinance-guide/>
- D. **Usando Webhooks**. 2023. Disponível em: <https://support.discord.com/hc/pt-br/articles/228383668-Usando-Webhooks>

Aroussi, R. **yfinance documentation**. Disponível em:
<https://ranaroussi.github.io/yfinance/index.html>